

# 双模型智能体协作开发教学案例

## 雪峰滑雪租赁管理系统：从需求、数据库到云端部署

一套可复现的 AI 辅助软件工程课堂案例

| 状态     | 项目角色   | 最后更新           |
|--------|--------|----------------|
| 已完成并部署 | 教学案例项目 | 2026 年 9 月 8 日 |

| 参与角色     | 配置                              |
|----------|---------------------------------|
| 执行智能体    | WorkBuddy，执行模型为 DeepSeek V4 Pro |
| 架构与审核智能体 | GPT：拆解阶段、生成提示词、独立审计与补漏          |
| 人类负责人    | 提出目标、确认业务取舍、控制凭据/写库/上线授权        |
| 适用范围     | 高职/本科软件工程、数据库、Web 开发、AI 工程实践课程  |

**文档性质** 本文件是依据 Git 历史、代码、数据库迁移、验收报告与残存对话摘要复原的教学案例。附录提示词是可复用的“教学复原版”，不是逐字聊天记录。

**项目地址（线上演示）**

<https://snowpeak-rental-d6fao36oc26a8db9-1453395692.tcloudbaseapp.com/>

# 目录

- 目录..... 2
- 1 案例概览 ..... 4
  - 1.1 学习目标..... 4
  - 1.2 最终成果..... 4
- 2 业务背景与需求基线..... 4
  - 2.1 核心业务对象 ..... 5
  - 2.2 角色权限..... 5
  - 2.3 设计中最重要五个决定 ..... 5
- 3 双模型智能体协作方法 ..... 6
  - 3.1 为什么用两个模型..... 6
  - 3.2 模型是否可以替换..... 6
  - 3.3 每轮交互的标准合同..... 7
- 4 Skill、工具与技术栈..... 7
  - 4.1 使用到的 Skill ..... 7
  - 4.2 工具链..... 7
  - 4.3 前端与云端技术栈..... 8
- 5 系统架构与安全边界..... 8
  - 5.1 local/cloud 双模式 ..... 9
  - 5.2 数据模型..... 9
  - 5.3 权限的四层防线 ..... 9
- 6 从零到上线的阶段流程 ..... 10
  - 6.1 为什么分阶段而不是一次完成..... 11
  - 6.2 阶段报告的固定格式..... 11
- 7 提示词工程：怎样让智能体像工程团队一样工作..... 11
  - 7.1 一条高质量工程提示词的结构..... 11
  - 7.2 核心原则..... 12
- 8 主要阶段提示词（教学复原版） ..... 12
  - P0 需求分析与设计基线..... 12
  - P1 本地 MVP 与模块化验证..... 13

|                                         |    |
|-----------------------------------------|----|
| P2 CloudBase PostgreSQL 迁移设计与权限加固 ..... | 13 |
| P3 迁移应用、Auth 建号与 UID 绑定 .....           | 14 |
| P4 云端只读迁移 .....                         | 14 |
| P5 普通 CRUD 云端写入 .....                   | 15 |
| P6 合同与维修 RPC 写入 .....                   | 15 |
| P7 驾驶舱、全链路 E2E、部署与文档 .....              | 16 |
| 9 可重复使用的审核与补漏提示词 .....                  | 16 |
| 10 代表性缺陷与工程教训 .....                     | 17 |
| 10.1 最有价值的测试不是“通过”，而是“失败得正确” .....      | 18 |
| 11 测试、验收与安全 .....                       | 18 |
| 11.1 自动化验证规模 .....                      | 18 |
| 11.2 真实 E2E 证据 .....                    | 19 |
| 11.3 安全规则 .....                         | 19 |
| 12 部署、运维与 Git 证据链 .....                 | 19 |
| 12.1 部署流程 .....                         | 19 |
| 12.2 关键 Git 里程碑 .....                   | 20 |
| 13 课堂实施方案 .....                         | 20 |
| 13.1 四次课建议 .....                        | 20 |
| 13.2 学生实践任务 .....                       | 21 |
| 13.3 评分量规（100 分） .....                  | 21 |
| 14 复现指南与限制 .....                        | 21 |
| 14.1 最小复现步骤 .....                       | 21 |
| 14.2 限制与诚实披露 .....                      | 21 |
| 14.3 项目资料索引 .....                       | 22 |
| 附录 A 验收命令清单 .....                       | 22 |
| 附录 B 术语表 .....                          | 23 |
| 附录 C 给教师的演示脚本 .....                     | 23 |

# 1 案例概览

本案例展示的不是“让 AI 一次生成整个系统”，而是把智能体放进一条可审计的软件工程流水线：先固化需求和数据规则，再逐阶段实现；每一阶段必须交付代码、测试、数据库证据和风险声明；另一模型独立审核，发现缺口后再生成补漏提示词。

## 1.1 学习目标

- 把自然语言需求转化为可验证的业务规则、数据模型、角色权限和验收标准。
- 理解双模型分工：执行智能体负责动手，审核智能体负责门禁、质疑与下一阶段提示词。
- 学会在 local 与 cloud 双模式下维护同一套 UI，同时避免云端失败时回退到本地假数据。
- 掌握 PostgreSQL RLS、列级权限、SECURITY DEFINER RPC、费率快照与状态机等关键设计。
- 建立安全意识：凭据不入仓库，真实 DML/部署/推送必须由人类明确授权。
- 用测试、浏览器验收、Git 里程碑和教学文档形成可复现证据链。

## 1.2 最终成果

| 成果         | 可验证结果                                             |
|------------|---------------------------------------------------|
| 双模式 Web 系统 | local 模式保留教学演示；cloud 模式使用真实 Auth、PostgreSQL 与静态托管 |
| 数据与权限      | 13 张业务表、RLS、列级 ACL、8 个业务/鉴权 RPC 与 1 个内部触发器函数      |
| 演示数据       | 13 表共 184 行，覆盖客户、设备、合同、换货、维修和排班                   |
| 自动化质量门禁    | 16 套验证/审计脚本，共 2,200 项检查或断言，全部通过                   |
| 真实 E2E     | 三角色登录、权限路由、驾驶舱、维修 RPC 生命周期、响应式与部署验收               |
| Git 证据     | 从初始版本到云端部署共 10 个关键提交，主分支已推送                       |

**课堂重点** AI 的价值不在于替代工程纪律，而在于把需求拆解、实现、验证和文档化的循环做得更快、更可追溯。

# 2 业务背景与需求基线

雪峰滑雪租赁系统服务门店管理员、店员和维修承包商。业务表面上是增删改查，真正的难点在设备状态、跨门店流转、合同金额、换货历史、维修费率快照与角色最小权限。

2.1 核心业务对象

| 对象      | 职责                 | 关键约束                                          |
|---------|--------------------|-----------------------------------------------|
| 客户与账号   | 客户资料、登录身份、员工/承包商关联 | 账号角色必须和关联实体精确匹配；UID 唯一                        |
| 门店与员工   | 归属门店、排班            | 排班时间范围、同日唯一、staff 权限受限                        |
| 设备与技能等级 | 一件实物一条记录           | home_store 与 current_store 分离；status 只由业务流程修改 |
| 合同与明细   | 出租设备、金额、归还         | 日租金快照；完成时间与明细状态一致                             |
| 换货记录    | 一出一进的合同变化          | 同一 change_group 两条记录，金额差可复算                   |
| 承包商与费率  | 维修主体与生效费率          | 费率按日期生效；被维修单引用后冻结                             |
| 维修单     | 待维修→维修中→已完成        | 本人操作；成本=工时×冻结费率；设备状态联动                        |

2.2 角色权限

| 模块       | admin    | staff    | contractor     |
|----------|----------|----------|----------------|
| 驾驶舱      | 读        | 读        | 禁止             |
| 客户       | CRUD     | 读/按策略开放  | 禁止             |
| 设备       | CRUD     | 只读       | 禁止             |
| 门店/员工/排班 | 管理       | 禁止       | 禁止             |
| 承包商/费率   | 管理       | 禁止       | 禁止             |
| 合同       | 读与业务 RPC | 读与业务 RPC | 禁止             |
| 维修单      | 全读、创建    | 全读、创建    | 只读本人、开始/完成本人维修 |

2.3 设计中最重要五个决定

- 实物设备不使用 quantity 堆叠，而是“一物一码、一件一行”，因此状态和门店位置可追踪。
- 设备归属门店与当前位置分开建模，支持跨店归还而不篡改资产归属。
- 合同明细保存 daily\_rate\_snapshot，维修单保存 rate\_id，历史金额不受未来调价影响。
- 换货不是覆盖旧明细，而是生成“归还+增加”两条成组变更，保留可审计历史。
- 云端写入不让页面直接拼复杂 SQL；合同与维修状态机由数据库 RPC 原子执行。

### 3 双模型智能体协作方法

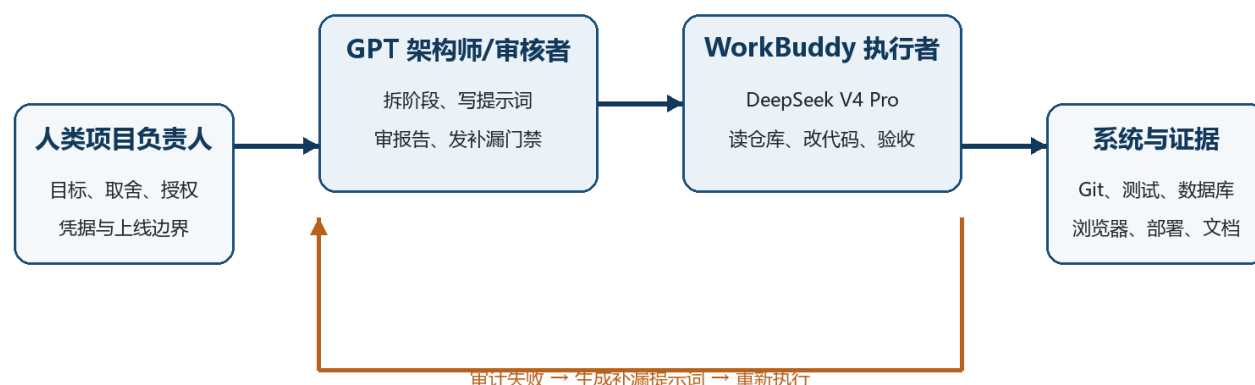


图 1 双模型协作闭环：执行与审核分离，人类保留授权权

#### 3.1 为什么用两个模型

单一智能体既写代码又评价自己的工作，容易把“能编译”误当作“业务正确”。本案例把角色拆开：GPT 先将目标写成带门禁的阶段提示词；WorkBuddy 依据提示词执行；GPT 再只读审核其报告、代码和证据。如果发现真实缺口，只下发范围明确的补漏任务。

| 角色         | 本案例配置                       | 主要职责                  | 不可越界             |
|------------|-----------------------------|-----------------------|------------------|
| 提示词架构师/审核者 | GPT                         | 拆阶段、写门禁、独立审查、提出补漏     | 不把执行者的自报结果直接当证据  |
| 执行智能体      | WorkBuddy + DeepSeek V4 Pro | 读仓库、修改、查询、测试、浏览器验收、报告 | 未获授权不做真实写库、部署、推送 |
| 人类负责人      | 教师/学生项目负责人                  | 确定需求、处理凭据、批准危险操作、最终验收 | 不把密码写进提示词/仓库/截图  |

#### 3.2 模型是否可以替换

可以。这里的关键是“角色与能力契约”，不是厂商名称。GPT 审核者可以替换为通义千问、Kimi、智谱 GLM、DeepSeek、豆包或其他国产智能体；WorkBuddy 的执行模型也可以替换。替换时至少要验证：能读取项目文件、能运行工具、能遵守只读/写入边界、能输出结构化验收证据、能在长任务中保持阶段状态。

**教学建议** 初学者先让一个模型执行、另一个模型审核；熟练后再尝试同一平台内的主代理+子代理。但无论几模型，都要保留独立验收与人类授权。

3.3 每轮交互的标准合同

| 输入                   | 执行                    | 输出                       |
|----------------------|-----------------------|--------------------------|
| 范围、基线文件、允许/禁止操作、验收命令 | 先读技能与基线；只读核验；实现；测试；审计 | 改动文件、关键设计、命令结果、未验证项、边界声明 |
| 审核问题或失败证据            | 最小范围修复；新增行为测试；全量回归    | 缺口根因、修复方式、测试增量、Git 状态    |
| 明确授权（写库/部署/推送）       | 先门禁检查，再执行一次，再只读验收     | 任务 ID/提交哈希/远端一致性/可恢复性    |

4 Skill、工具与技术栈

4.1 使用到的 SKILL

| Skill         | 作用                  | 在案例中的具体使用                          |
|---------------|---------------------|------------------------------------|
| spec-workflow | 把需求写成可验收规范          | 读取需求、业务规则和验收标准；控制阶段范围；补漏时锁定缺口      |
| ui-design     | 保持统一界面系统            | 延续雪线主题、TDesign 组件、桌面端响应式布局与只读/可写状态 |
| 说明            | Skill 是给智能体的结构化工作说明 | 它不是业务代码；不同平台可用系统提示词、规则文件或工作流模板替代   |

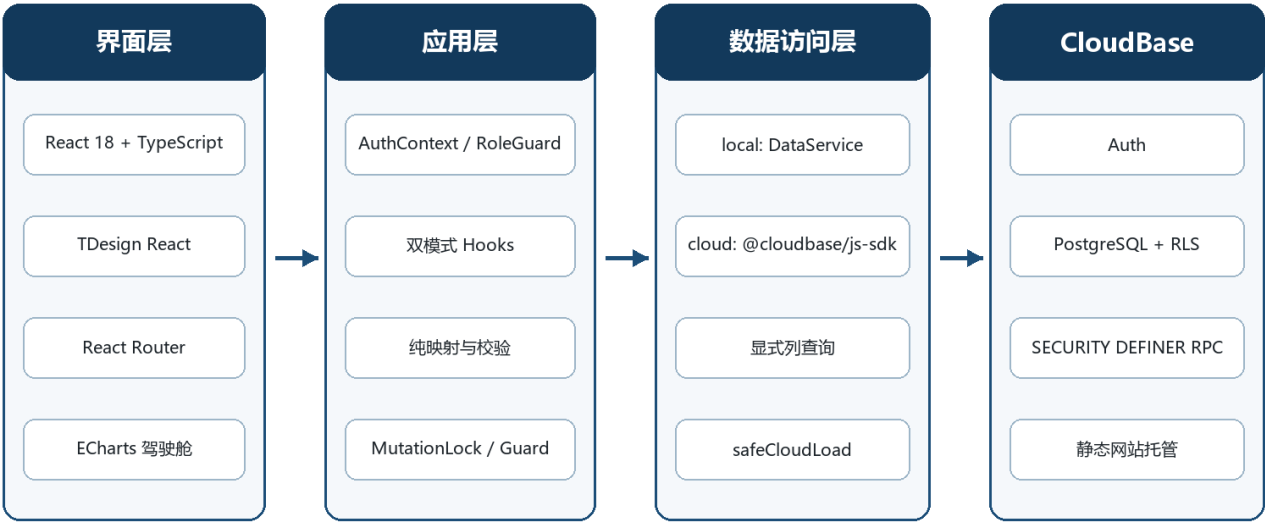
4.2 工具链

| 类别    | 工具                                 | 用途                                     |
|-------|------------------------------------|----------------------------------------|
| 代码与版本 | 文件读写、终端、Git/GitHub                 | 实现、差异审计、里程碑提交和远端核对                     |
| 数据库   | CloudBase MCP / PostgreSQL 查询与迁移工具 | 系统目录核验、plan/apply migration、ACL/RLS 验收 |
| 浏览器   | 浏览器自动化                             | 三角色登录、路由权限、CRUD/RPC、响应式、控制台错误          |
| 质量    | TypeScript、ESLint、Vite、Node 验证脚本   | 静态类型、风格、构建与行为级单测                       |
| 交付    | CloudBase 静态托管、Markdown/DOCX/PDF   | 上线、验收报告、教学材料                           |

4.3 前端与云端技术栈

| 层         | 技术                                 | 版本/说明                                        |
|-----------|------------------------------------|----------------------------------------------|
| 前端框架      | React + TypeScript + Vite          | React 18.3.1; TypeScript 5.5.4; Vite 5.4.3   |
| UI/路由/图表  | TDesign React、React Router、ECharts | TDesign 1.10.1; Router 6.26.2; ECharts 5.5.1 |
| Cloud SDK | @cloudbase/js-sdk                  | 3.9.0; Auth 与 PostgREST 风格 RDB 客户端           |
| 数据库       | CloudBase PostgreSQL               | RLS、列级 ACL、约束、触发器、SECURITY DEFINER<br>RPC    |
| 部署        | CloudBase 静态网站托管                   | 生产构建 + SPA 回退                                |
| 本地模式      | DataService + localStorage         | 用于课堂演示与无云环境回归，不作为多用户安全边界                     |

5 系统架构与安全边界



VITE\_DATA\_MODE = local / cloud; 非法或缺失云配置时 fail-closed, 不静默回退

图 2 双模式技术架构



### 5.1 LOCAL/CLOUD 双模式

同一套页面通过 VITE\_DATA\_MODE 选择数据源。local 用 DataService 和 localStorage；cloud 用 CloudBase Auth 和 PostgreSQL。关键原则是 fail-closed：用户明确选择 cloud 后，只要环境 ID、区域或访问密钥缺失/非法，应用显示阻塞页，绝不静默降级为 local，也不展示 demo123 快捷账号。

| 场景                         | 预期行为                                       |
|----------------------------|--------------------------------------------|
| VITE_DATA_MODE 缺失/空或 local | 进入 local；保留演示账号与重置入口                       |
| cloud 且配置完整                | 初始化 CloudBase；恢复真实会话；查询 accounts 建立业务身份    |
| cloud 且缺配置                 | 配置错误页；不挂载 DbProvider/AuthProvider，不碰本地业务数据 |
| 非法 dataMode                | 配置错误页；不回退 local                            |
| 云查询失败                      | 显示安全错误和重试；不回退本地种子数据                        |

### 5.2 数据模型

| 域    | 表                                                  | 说明                             |
|------|----------------------------------------------------|--------------------------------|
| 身份   | accounts                                           | CloudBase UID 与业务角色、员工/承包商实体绑定 |
| 基础资料 | skill_levels, stores, customers, employees         | 技能等级、门店、客户、员工                  |
| 资产   | rental_items                                       | 一件一行，含归属门店、当前位置与状态             |
| 供应商  | contractors, contractor_rates                      | 承包商与按日期生效的小时费率                 |
| 合同   | rental_contracts, contract_lines, contract_changes | 合同头、设备明细、换货/归还审计轨迹             |
| 维修   | repair_orders                                      | 设备维修状态机与冻结费率                   |
| 人力   | shifts                                             | 员工门店排班                         |

### 5.3 权限的四层防线

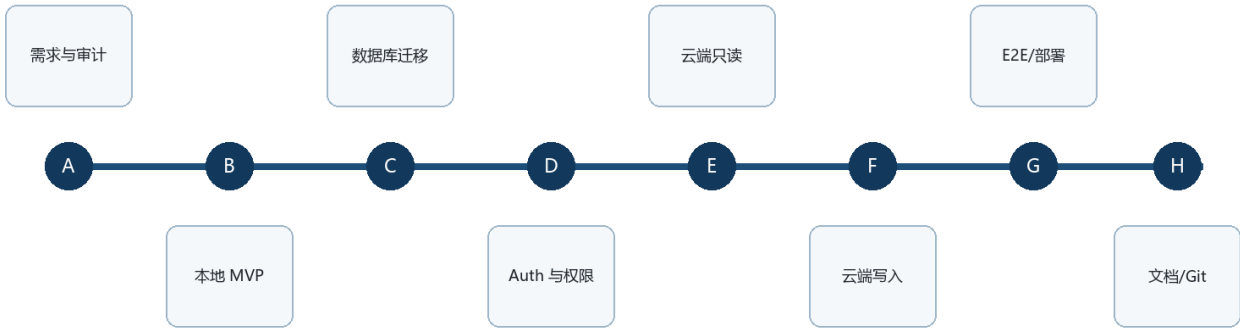
- 路由与页面：RoleGuard 隐藏无权限模块，cloud 只读阶段隐藏写入口。
- Hook/分派层：角色与字段必须在 getRdb 之前校验，避免未授权请求触达云端。
- PostgreSQL：RLS 决定可见行，列级 ACL 封闭 item\_id/status 等系统字段。
- RPC/触发器：函数体重新读取 auth.uid()、accounts.enabled 与角色；复杂状态机原子执行；已引用费率由触发器冻结。



关键设计：一件设备一条记录 | home\_store 与 current\_store 分离 | 费率冻结 | 换货双记录成组

图 3 合同与维修业务主链路

6 从零到上线的阶段流程



每阶段都遵循：读取基线 → 只读核验 → 实现 → 自动测试 → 浏览器/云端验收 → 审计 → Git 里程碑

图 4 A—H 阶段路线图

| 阶段       | 主要工作                      | 门禁证据             | 退出条件        |
|----------|---------------------------|------------------|-------------|
| A 需求与审计  | 需求、规则、数据模型、UI 规范          | 冲突清单、业务规则编号、验收矩阵 | 设计基线可被实现和测试 |
| B 本地 MVP | localStorage 数据服务、页面、角色演示 | 模块验证脚本、浏览器回归     | 离线可完整演示     |

| 阶段         | 主要工作                    | 门禁证据                           | 退出条件                 |
|------------|-------------------------|--------------------------------|----------------------|
| C 数据库迁移    | Schema、Seed、RLS、RPC、ACL | planMigration、静态审计、哈希          | 迁移可执行且未越权            |
| D Auth 与权限 | 真实账号、UID 绑定、SDK 初始化     | 三账号 ACTIVE、映射唯一、配置 fail-closed | 真实登录/刷新/退出通过         |
| E 云端只读     | 客户→主数据→人力→合同→维修→驾驶舱     | 显式列、严格映射、零本地回退                 | 三角色只读页面正确            |
| F 云端写入     | 普通 CRUD 与 RPC 写操作       | 前置校验、锁、刷新、错误映射                 | fake RDB + 真实 DML 验收 |
| G E2E 与部署  | 三角色、响应式、控制台、静态托管        | 真实浏览器和只读回查                     | 线上关键链路通过             |
| H 文档与 Git  | README、E2E 报告、教学指南、提交   | 敏感扫描、提交哈希、远端一致                 | 可复现交付                |

6.1 为什么分阶段而不是一次完成

大型提示词一次塞入需求、数据库、UI、测试和部署，会产生三个问题：上下文失焦、错误难定位、授权边界模糊。阶段化让每次修改都有稳定基线；云端真实写入与部署被推迟到静态验证和只读核验全部通过之后。

6.2 阶段报告的固定格式

- 完成内容：文件、表、RPC、页面与行为。
- 关键设计：nullable、状态、权限、错误映射、并发与卸载安全。
- 验收证据：命令结果、数据库只读查询、浏览器观察。
- 未验证项：不能因为缺密码或未执行真实 DML 就写成已通过。
- 边界声明：未修改哪些对象、未执行哪些危险操作、Git 是否提交。

7 提示词工程：怎样让智能体像工程团队一样工作

7.1 一条高质量工程提示词的结构

| 组成    | 问题         | 写法示例                                            |
|-------|------------|-------------------------------------------------|
| 身份与范围 | 谁来做、这批只做什么 | “进入第 4 批，只迁移合同列表与详情只读，不做写入”                     |
| 强制读取  | 以什么为事实源    | 列出 schema、RLS、docs、types、页面、既有模式                |
| 前置核验  | 哪些事实不能猜    | 先查真实列、nullable、RLS、SDK 返回格式                     |
| 实现约束  | 必须怎样写      | 显式列、fail-closed、cloud 零 local reader、固定 Hook 顺序 |
| 验收命令  | 怎样证明完成     | 指定 validate/typecheck/lint/build/audit          |
| 危险操作  | 哪些需要授权     | 本轮禁止 apply/DML/建用户/部署/commit/push               |

| 组成   | 问题        | 写法示例            |
|------|-----------|-----------------|
| 报告格式 | 如何让下一模型审核 | 改动、结果、未验证项、边界声明 |

7.2 核心原则

- 把“完成”改写为可观测状态，例如“getRdb 调用 0 次”“影响行数恰好 1”“远端 HEAD 与本地一致”。
- 要求智能体先读再改；但不要让它无限探索，明确必读文件与退出条件。
- 把安全边界写成否定约束：不读密钥、不打印 Token、不执行真实 DML、不提交 .env.local。
- 行为测试优先于源码字符串匹配；测试应驱动真实纯函数/分派/状态机，避免 check(true)。
- 审核提示词只修真实缺口，禁止顺手扩大范围；补漏完成后重新跑全量回归。

**是否只保留开头提示词？** 不建议。开头提示词能展示构思，但学生学不到如何把系统安全地推进到云端。本案例保留每个“大阶段”的完整提示词；几十轮重复补漏则抽象成通用模板，兼顾完整性与可读性。

8 主要阶段提示词（教学复原版）

以下提示词依据项目源码、Git 提交、迁移文件、验收报告与残存摘要重构。使用时把【项目路径】【环境 ID】【阶段编号】替换为自己的信息。真实密码、Access Key、Token 永远不要放入提示词；需要登录时由人类在安全界面或被忽略的本地环境文件中输入。

P0 需求分析与设计基线

可复制提示词

你是本项目的执行智能体。先加载 spec-workflow 与 ui-design 两个 skill，完整读取【项目路径】中的需求、现有代码、数据库课程背景材料和界面参考。当前阶段只做需求与设计审计，不写云端数据、不部署、不提交。

目标：形成可直接指导实现的需求基线。请完成：

1. 提取角色、用例、业务规则、验收标准，并给规则编号；指出需求之间的矛盾和隐含假设。

2. 建立概念数据模型与关系：客户、账号、门店、员工、设备、技能等级、承包商、费率、合同、合同明细、换货变更、维修单、排班。

3. 明确关键决策：一件设备一行；home\_store/current\_store 分离；合同日租金快照；维修费率冻结；换货用同组两条变更记录；状态机由服务层或 RPC 控制。

4. 输出页面信息架构、角色可见性、桌面端 1280/1440/1920 响应式要求与统一视觉规范。

5. 生成阶段 0-2 的实现清单和验收矩阵，但不要开始编码。

报告必须列出：事实来源、冲突与取舍、数据实体、业务规则、角色矩阵、页面清单、风险、下一阶段入口条件。任何不确定项明确标注，不得自行虚构。

**应提交的证据：**需求基线、数据字典、角色矩阵、页面规范、风险与验收矩阵。

## P1 本地 MVP 与模块化验证

### 可复制提示词

进入本地 MVP 实现阶段。先读取已批准的需求、数据设计、系统设计和界面规范；加载 spec-workflow 与 ui-design。仅实现 local 模式，不接云端、不改数据库、不部署、不提交。

实施要求：

1. 使用 React + TypeScript + Vite + TDesign，建立统一 AppLayout、登录、RoleGuard、403 与业务路由。
2. DataService 以 localStorage 为持久层，封装种子、版本迁移、订阅刷新、角色权限、输入校验和业务状态机；页面不得直接散写 localStorage。
3. 按主数据→客户→员工排班→合同→维修→驾驶舱顺序实现。合同必须支持新建、换货、归还；维修必须支持创建、开始、完成。
4. 保持“一物一码”、门店位置、费率快照、金额复算与状态联动；错误必须给安全且可理解的中文提示。
5. 为每个域编写 Node 行为验证脚本，覆盖正向与反向用例；运行 typecheck、lint、build；浏览器验证三角色与三档宽度。

每完成一个模块先报告并等待审核，不得越过当前模块。报告包含文件、规则、测试数量、浏览器结果、遗留风险与未执行操作。

**应提交的证据：**本地可运行系统、域验证脚本、浏览器证据、无云端副作用。

## P2 CLOUDBASE POSTGRESQL 迁移设计与权限加固

### 可复制提示词

进入 CloudBase PostgreSQL 迁移设计阶段。先加载两个 skill，读取 schema/业务规则/本地 DataService，并只读核验【环境 ID】的实例、current\_user、public schema、现有对象与迁移历史。本阶段只允许 planMigration 与只读查询，禁止 applyMigration、DDL/DML、Auth 用户操作和部署。

请生成有序 migration：

1. create\_schema：13 张业务表、主键/外键/UNIQUE/CHECK/identity，字段 nullable 必须与业务一致。
2. create\_rls\_policies：启用 RLS；定义 admin/staff/contractor 读写策略；实现 current\_app\_role 与合同/换货/归还/维修 RPC；函数体用 auth.uid() 关联 accounts，校验 enabled 和角色；SECURITY DEFINER 固定 search\_path。
3. seed\_demo\_data：按全部外键拓扑排序插入 184 行种子，并校准 15 个序列。
4. harden\_cloudbase\_acl：盘点 aclexplode、序列、函数、schema 与 pg\_default\_acl；撤销 anon/authenticated 的宽泛默认权限，按最小矩阵重授权；service\_role 保留；内部触发器函数不授 authenticated。

同时新增 migration 静态审计：锁定已应用文件哈希、检测未知 migration、逐函数检查鉴权、检查列级权限不暴露 rental\_items.status/item\_id。依次运行 audit、seed、typecheck、lint、build，再逐个 planMigration。明确声明 executable=true 只是预检，不等于真实 PostgreSQL 语义验收。

**应提交的证据：**4 个 migration、ACL 矩阵、审计脚本、planMigration 结果与零写入声明。

## P3 迁移应用、AUTH 建号与 UID 绑定

### 可复制提示词

进入 CloudBase 迁移执行与 Auth 接入门禁阶段。所有真实写入必须“先核对、执行一次、只读验收”。不得打印密码、Access Key 或 Token。

步骤 A：重新核对 migration SHA256、审计全绿、listMigrations 状态、current\_user/session\_user 与对象 owner；planMigration 无冲突后，仅调用一次 applyMigration。拿到明确终态后只读验证迁移历史、13 表 RLS、184 行、序列/函数/触发器/ACL/default ACL。

步骤 B：只读核验 CloudBase Auth 登录方式和 Publishable/Access Key 配置。人类在控制台手工创建 admin、staff、contractor 三个 username/password 用户并保管密码；智能体只读取用户名、UID、状态。

步骤 C：回填前校验三 Auth 用户 ACTIVE、UID 非空互异；accounts 三行角色/实体关联正确且 uid 全 NULL。执行单个原子 DO 块，按 account\_id+username+role+enabled+关联字段+uid IS NULL 精确更新，每条 ROW\_COUNT 必须为 1，任何异常整块回滚。随后只读验证唯一映射、总行数和迁移不变。

步骤 D：安装并锁定 @cloudbase/js-sdk。新增 .env.example 占位，.env.local 必须忽略。实现 cloudbase 惰性单例、getSession 恢复、账号唯一行加载、disabled/角色异常立即 signOut；cloud 模式隐藏 demo 快捷与重置入口。配置解析必须 fail-closed，错误时不挂载本地数据 Provider。

最后执行 auth 测试、全量回归和 local/cloud/非法配置浏览器场景。未获下一条授权，不进入业务查询。

**应提交的证据：**apply 任务终态、ACL/RLS 实测、三账号 UID 映射、Auth 双模式与 fail-closed 证据。

## P4 云端只读迁移

### 可复制提示词

进入 CloudBase 业务查询迁移。采用小批次：客户→门店/技能/设备→承包商/费率→员工/排班→合同列表与详情→维修单→驾驶舱。每批只迁移指定页面，禁止写入、改数据库、部署和提交。

每批统一执行：

1. 只读核验真实表列、nullable、RLS、行数及 SDK 返回格式；禁止凭 TS 类型猜 bigint/numeric/date/timestamp。
2. 建立 cloudXxx 纯逻辑层：显式 select 列（禁 \*）、字符串数值归一化、正安全整数、严格日期/时间、nullable 保留 null、重复主键和关联完整性 fail-closed。
3. 建立 DataSource 分派与 safeCloudLoad，收口 getRdb 同步 throw、Promise reject、SDK error；失败返回空数据+安全错误，不泄露底层细节、不回退 localStorage。
4. Hook 在 cloud 下用 useDbData(enabled:false, disabledValue) 保证零本地 reader、零订阅；使用请求代次与卸载防护。
5. 页面提供 loading/error/retry/empty；cloud 显示“CloudBase PostgreSQL”并按阶段隐藏写入口；local CRUD 不回归。
6. 合同详情严格解析路由 ID；合同做时间、状态、换货组和金额复算；contractor 维修列表只能调用 list\_my\_repairs RPC，不读无权引用表。

为每批写可注入 fake RDB 的行为测试，断言精确 from/select/order、异常收口、零本地回退和角色路径。全量 typecheck/lint/build/audit 后停止，列出需要真人登录验证的项目。

**应提交的证据：**每个模块的只读视图模型、查询 Hook、行为测试、角色与错误场景证据。

## P5 普通 CRUD 云端写入

### 可复制提示词

进入 CloudBase 普通 CRUD 迁移。按客户→设备→门店→承包商/费率→员工/排班逐批推进；每批先只读核验真实 ACL/RLS/约束，再实现，不执行真实 DML，直到静态审核通过且人类授权。

统一写入约束：

1. 独立 cloudXxxMutations：输入模型明确 nullable；payload 白名单，绝不包含 identity/status/uid/role 等系统字段；字段与目标 ID 在 getRdb 前校验，mutation 内再次校验。
2. cloud 权限在分派层提前门禁，例如只有 admin 可写时，staff/contractor 调 Hook 也必须 getRdb/cloudRun/from 全 0 次；后端 RLS 是最终防线但不是唯一防线。
3. 使用 MutationLock 阻止重复提交；成功后先 guard.begin() 立即失效旧查询，再重读；失败/local/卸载不刷新；卸载后不 setState、关框或 Message。
4. insert/update/delete 影响行数必须恰好 1；数据库错误按 operation+SQLSTATE 映射为安全中文文案。触发器 P0001 只在适用的 update/delete 场景映射，其他操作不得误报。
5. 页面 local/cloud 共用布局；cloud 根据角色显示写入口；按钮 loading 直接使用 Hook mutating；输入组件只改善 UX，服务层校验才是门禁。

测试必须用计数型 getRdbFn/cloudRun/localRun/from，证明非法 ID、字段、非 admin 在云连接前被拒；覆盖安全整数、null/空串、枚举、日期、唯一/外键/引用错误、锁与刷新代次。先提交静态报告，等待是否跳过或执行真实云端 DML 的明确指令。

**应提交的证据：**CRUD mutation 层、前置权限/校验、锁与刷新、错误映射、计数型测试。

## P6 合同与维修 RPC 写入

### 可复制提示词

进入复杂业务写入阶段，只迁移数据库已提供的 SECURITY DEFINER RPC，不在前端复刻多表事务。先读取 RPC 签名、函数体、GRANT、RLS、CHECK/外键和页面 local 流程；只读核验远端函数存在与 authenticated EXECUTE。

合同：调用 create\_contract、exchange\_item、return\_items。严格校验合同 ID、客户/员工/门店/设备、日期与选中明细；create 只能选择在库设备；换货必须旧设备借出中、新设备在库；归还支持单件/批量。RPC 成功后重读合同详情与列表，不本地伪改状态。

维修：admin/staff 调 create\_repair\_order；contractor 只能 start\_repair/complete\_repair 本人维修单。contractor 查询继续只走 list\_my\_repairs。完成时工时必须正数且 0.25 倍数；费用由数据库按 rate\_id 冻结费率计算，前端只展示返回结果。

所有分派先做角色和 ID/字段校验，再 getRdb；MutationLock、请求代次、卸载防护与安全错误映射沿用普通 CRUD。RPC error/reject/sync throw 全收口，不回退 local。测试覆盖函数名/参数、角色调用 0 次、非法状态、并发与刷新。

静态审核后，如人类明确授权真实 DML，选择可清理的测试数据执行一次完整生命周期；每一步读回验证，最后删除 E2E 数据并确认 13 表总行数恢复 184。

**应提交的证据：**RPC 调用层、状态机证据、真实生命周期读回与基线恢复（若获授权）。



## P7 驾驶舱、全链路 E2E、部署与文档

### 可复制提示词

进入收尾阶段。先确认 A-F 已通过静态审计，真实密码只由人类在安全会话中提供，绝不写文件/日志/截图/Git。若发现凭据曾进入对话，完成后提示轮换。

1. 驾驶舱云端化：显式查询所需表，严格映射 date/timestamp/numeric，KPI 失败不显示伪数据；校验利用率、营收、维修周转、门店分布。
2. 浏览器 E2E：分别登录 admin/staff/contractor，验证刷新恢复、退出、路由 403、各模块行数 and 写入口；采集 console/pageerror；验证 1280/1440/1920 无 document 横向溢出。
3. 真实业务：在获授权范围内验证客户 CRUD 和维修 create→start→complete；每次写后查询数据库；最后清理测试数据并核对 184 行基线。
4. 构建与部署：cloud 模式生产 build，上传静态托管，设置 SPA fallback；线上通过风险提示后再做 admin 登录与驾驶舱冒烟。
5. 文档：更新 README、云端 E2E 报告、教学指南，明确 local/cloud、账号分发、环境变量、测试命令、权限边界与已知限制。
6. Git：敏感扫描、.env.local/.workbuddy/dist/node\_modules 排除、diff 范围审核、分里程碑 commit，非 force 推送；用 ls-remote 核对远端 main 与本地 HEAD。

最终报告不得只说“完成”，必须给出 E2E 数据、清理结果、部署 URL、命令结果、提交哈希、远端一致性和未验证项。

**应提交的证据：**线上系统、三角色 E2E、基线恢复、文档、提交与远端一致性。

## 9 可重复使用的审核与补漏提示词

实际开发中，大量对话来自审核后的补漏。没必要逐字保存每一轮；应保存“能稳定发现问题的模板”，并把本轮的具体文件、函数和验收标准代入。

### T1 · 独立静态审核

请作为独立审核者，只读检查【阶段名称】的实现与执行报告。不要先接受报告结论，也不要扩大范围。

逐项核验：

- Git diff 是否只包含声明文件；未跟踪文件是否被漏审；
- 数据库列/nullable/RLS/ACL/RPC 是否与真实远端一致；
- cloud 失败是否 fail-closed，是否存在 localStorage 回退；
- 字段、目标 ID、角色是否在 getRdb 前拒绝；
- payload 是否只含白名单列；影响行数是否恰好 1；
- MutationLock、请求代次、卸载防护和错误映射是否覆盖所有路径；
- 测试是否真实驱动行为，是否存在 check(true)、源码字符串匹配或虚假声明；
- 未执行真实 DML 时是否明确声明。

如果没有真实缺口，输出“审核通过”与证据；如果有，逐项给出根因、影响、最小修复范围、必须新增的行为测试和全量回归命令。不要直接提交或推送。



T2 • 权限/前置校验补漏

进入【模块】补漏。只修以下缺口：【缺口列表】。先读取分派、Hook、mutation、页面和测试。

要求：cloud 模式对角色、目标 ID 与字段做同步前置校验；失败时 getRdb/cloudRun/from/localRun 全为 0；mutation 内保留二次校验。用计数型 fake 依赖新增测试，覆盖合法/非法/非 admin/异常；不要用源码字符串匹配。MutationLock 必须 finally 释放；成功刷新先失效旧 token；卸载后不推进代次、不 setState、不 Message。保持 local 行为与 Hooks 顺序。

运行本模块测试、全量 validate、audit、typecheck、lint、build、git diff --check。报告只列本批改动和未验证项，禁止 DML/数据库变更/commit/push。

T3 • 真实云端 DML/E2E 门禁

进入真实云端验收。先只读确认目标账号、角色、数据基线、对象权限与测试数据可清理；确认全部静态命令通过。若任一门禁失败，立即停止。

仅执行【被授权的业务动作】，优先通过真实 SDK/RPC，而不是手写 SQL。每个动作：记录前状态→执行一次→读取数据库和 UI 验证→记录后状态。任何失败不得盲目重试写入。完成后用业务逆操作或精确清理恢复基线，再只读核对每表行数、状态、迁移历史与无残留。

密码/Access Key/Token 不写文件、不打印、不截图、不提交；报告用账号角色代替密码。输出真实请求链、返回状态、数据库证据、清理证据、未验证项。

T4 • Git 安全里程碑

保存【阶段名称】到 Git。先只读核对分支、origin、HEAD、status、diff --stat 和所有未跟踪文件。只提交【精确文件列表】。

安全门禁：

- 1. .env.local、密钥、密码、Token、session、截图、.workbuddy、dist、node\_modules、缓存和临时文件不得入库；
- 2. migrations 若不在本阶段范围不得变化；
- 3. 运行指定全量测试、typecheck、lint、build、git diff --check，任一失败停止；
- 4. 使用显式 git add 文件列表，随后再次核对 staged diff；
- 5. 以【commit message】提交；普通 push 到 origin/main，禁止 force、禁止改写历史；
- 6. 用 git ls-remote 核对远端 main 与本地 HEAD 完全一致。

报告 commit hash、文件范围、测试、安全扫描、push 输出和远端一致性。

10 代表性缺陷与工程教训

| 缺陷/风险               | 为什么容易出现        | 本案例的修复                         | 可迁移教训              |
|---------------------|----------------|--------------------------------|--------------------|
| cloud 配置失败却回退 local | 把模式解析和配置校验混在一起 | 拆分解析；配置门禁置于 DbProvider 外；非法值阻塞 | 安全模式必须 fail-closed |

| 缺陷/风险               | 为什么容易出现              | 本案例的修复                                       | 可迁移教训                |
|---------------------|----------------------|----------------------------------------------|----------------------|
| 云 Hook 仍读本地数据       | 为遵守 Hooks 顺序而无条件订阅   | useDbData enabled=false + disabledValue 联合类型 | 固定 Hook 顺序不等于必须执行副作用 |
| getRdb 早于字段/角色校验    | 校验被放在 mutation 内部    | 分派层前置校验，内部保留纵深防护                             | 用调用次数证明边界，而非只测错误文案   |
| 卸载后 mutation 仍刷新    | 异步结果晚到，setTick 无活跃判断 | isActive + mountedRef + guard + cancelled    | 状态安全需要生命周期和请求代次双重控制  |
| P0001 被统一解释为费率冻结    | 错误映射忽略 operation     | 仅 update/delete 将 P0001 映射为引用冻结              | 数据库错误码必须结合操作上下文      |
| timestamp 当 date 解析 | 真实 SDK 返回格式与 TS 假设不同 | 只读取样；timestamp 先严格解析/取日期                     | 云数据格式要实测，不能凭印象       |
| 设备 status 可被普通更新    | 表级 UPDATE 权限过宽       | 撤表级写权；列级授权排除 status；RPC 改状态                  | 业务状态字段应封闭            |
| 历史费率被修改             | 维修单只存当前金额或动态查费率      | 保存 rate_id；引用后触发器阻止改删                        | 历史计费必须可重放            |
| 换货覆盖旧记录             | 追求简化 CRUD            | 两条 contract_changes 共享 group_id              | 审计型业务优先追加事实，不覆盖历史    |

10.1 最有价值的测试不是“通过”，而是“失败得正确”

本案例的测试大量覆盖拒绝路径：非法 ID 不连接 RDB、非 admin 不调用 cloudRun、云查询失败不回退 local、旧请求晚到不能覆盖新 ID、卸载后不刷新、SDK 细节不出现在用户文案。这些反向断言比“页面能打开”更能说明系统边界。

11 测试、验收与安全

11.1 自动化验证规模

| 验证套件                                     | 检查数               |
|------------------------------------------|-------------------|
| seed（正向 1 + 反向 12）                       | 13                |
| master / workforce / contracts / repairs | 50 / 52 / 81 / 48 |
| auth                                     | 41                |
| cloud-read（客户）                           | 213               |
| cloud-master-read（设备/门店/主数据）             | 386               |
| cloud-workforce-read                     | 174               |
| cloud-contract-read / cloud-repair-read  | 197 / 142         |

| 验证套件                                     | 检查数       |
|------------------------------------------|-----------|
| cloud-contractor-write / workforce-write | 175 / 188 |
| cloud-contract-write / repair-write      | 90 / 88   |
| cloud-dashboard                          | 56        |
| audit:migrations                         | 206       |
| 合计                                       | 2,200     |

11.2 真实 E2E 证据

- admin、staff、contractor 三账号均完成登录、刷新恢复和退出。
- 角色路由和 403 符合矩阵；contractor 只看到本人 4 条维修单。
- 驾驶舱真实 KPI：利用率 47.1%、营收 ¥0.00、平均维修周转 1.8 天。
- 维修 RPC 完整链：创建→开始→完成；0.5 小时×冻结费率 ¥200=¥100；设备恢复“在库”。
- E2E 新增记录已精确清理，13 表恢复 184 行。
- 1280/1440/1920 三档视口通过，控制台无业务错误。

11.3 安全规则

| 风险            | 规则                                                        |
|---------------|-----------------------------------------------------------|
| 真实凭据          | 只在本机被忽略的环境文件或安全登录界面使用；不进提示词、源码、报告、截图、Git                  |
| 前端 Access Key | 仅使用平台允许公开的客户端令牌；数据库安全仍依赖 Auth、RLS、ACL、函数体鉴权               |
| 真实写库          | 必须获人类明确授权；先门禁，执行一次，随后只读验收与清理                              |
| SQL/RPC       | 复杂多表状态迁移走原子 RPC；SECURITY DEFINER 固定 search_path 并撤 PUBLIC |
| Git           | 显式文件清单、敏感扫描、禁止 force、远端哈希核对                               |
| 日志            | 用户错误文案不泄露 SQL、约束名、details、hint、Token                      |

12 部署、运维与 Git 证据链

12.1 部署流程

- 生产构建使用 cloud 配置；构建产物上传 CloudBase 静态托管。
- 配置 SPA 404 回退到 index.html，确保刷新深层路由不报 404。

- 线上首次访问通过平台风险提示后，验证 admin 登录和驾驶舱冒烟。
- 部署后仍以数据库只读查询、浏览器 E2E 和控制台错误为准，不以“上传成功”代替验收。

12.2 关键 Git 里程碑

| 提交      | 日期         | 内容                                       |
|---------|------------|------------------------------------------|
| ae29ef6 | 2026-08-27 | Initial commit                           |
| fedc71c | 2026-08-28 | CloudBase PostgreSQL migrations 与 ACL 审计 |
| c4ad773 | 2026-09-06 | Auth 与云端只读业务数据                           |
| 324da9f | 2026-09-06 | 客户 CRUD                                  |
| 7776b10 | 2026-09-07 | 设备 CRUD                                  |
| a4f6223 | 2026-09-07 | 门店 CRUD                                  |
| 2e333e6 | 2026-09-08 | 云端写入层与驾驶舱                                |
| 6dfe5ed | 2026-09-08 | 写操作校验与角色门禁                               |
| f1cedf2 | 2026-09-08 | E2E、教学指南与部署说明                            |

**可追溯性** Git 历史说明系统是逐阶段演进的。教学中应让学生解释每个里程碑新增了什么安全边界，而不是只展示最终页面。

13 课堂实施方案

13.1 四次课建议

| 课次    | 主题         | 课堂活动                        | 课后产出                   |
|-------|------------|-----------------------------|------------------------|
| 第 1 次 | 需求→规则→模型   | 两模型角色扮演；审查冲突；画数据关系          | 需求基线、角色矩阵、5 条关键设计决定    |
| 第 2 次 | 本地 MVP 与测试 | 让执行智能体完成一个模块；审核者找虚假测试       | 可运行 local 模块、行为测试、审计报告 |
| 第 3 次 | 云数据库与安全    | 设计 RLS/ACL/RPC；比较前端权限与数据库权限 | migration 预览、权限矩阵、风险说明 |
| 第 4 次 | E2E、部署与复盘  | 三角色验收；真实写入门禁；Git 里程碑演示      | E2E 证据、线上演示、个人反思       |

### 13.2 学生实践任务

- 替换业务域：把滑雪租赁改为实验室器材、图书借阅或车辆租赁，保留状态机和双模型流程。
- 新增“设备报废审批”，要求学生同时修改规则、Schema、RLS、RPC、UI 和反向测试。
- 故意引入一个 fail-open 缺陷，让审核模型通过调用次数测试定位。
- 比较同一提示词在 GPT、通义千问、Kimi、GLM、DeepSeek 等模型上的执行差异。
- 提交一份“AI 生成内容责任说明”：哪些决定由人类确认，哪些结果有独立证据。

### 13.3 评分量规（100 分）

| 维度       | 分值 | 优秀表现                           |
|----------|----|--------------------------------|
| 需求与数据模型  | 20 | 规则可追溯，约束进入 Schema/RPC/测试；无关键矛盾 |
| 提示词与任务分解 | 15 | 范围、门禁、证据、禁止项清晰；阶段可独立验收         |
| 实现质量     | 20 | 边界清楚、无重复业务逻辑、nullable 与状态正确    |
| 权限与安全    | 20 | 前端+Hook+RLS+ACL+RPC 多层防护；无凭据泄露 |
| 测试与 E2E  | 15 | 正反用例、失败收口、角色、真实读回与清理证据充分       |
| 文档与复盘    | 10 | 如实区分自动/手动、已验证/未验证、原始/复原提示词     |

## 14 复现指南与限制

### 14.1 最小复现步骤

1. 克隆仓库，进入 app，安装锁定依赖。
2. local：保持 VITE\_DATA\_MODE=local，运行开发服务器，使用演示账号完成流程。
3. cloud：复制 .env.example 为 .env.local，在本机填写环境 ID、region 与客户端访问令牌；确认该文件被 Git 忽略。
4. 按 migration 版本顺序 plan，再在独立环境 apply；创建 Auth 用户并安全绑定 accounts.uid。
5. 运行 package.json 中全部 validate:\*、audit:migrations、typecheck、lint、build。
6. 用三个角色完成浏览器验收；真实写入后清理测试数据并核对行数。

### 14.2 限制与诚实披露

- 原 WorkBuddy 长对话已经压缩，无法恢复逐字提示词；本文件提供的是证据驱动的教学复原版。
- localStorage 只用于教学和单机演示，不等同于真实多用户安全后端。
- 前端测试不能替代 PostgreSQL 约束、RLS、ACL 与函数体鉴权。
- 客户端 Access Key 是否可公开取决于平台定义；任何具有服务端特权的 Secret 都不得进入浏览器构建。

- 线上测试域名和平台登录策略可能随环境变化，复现时应重新只读核验。

14.3 项目资料索引

| 资料                        | 作用                       |
|---------------------------|--------------------------|
| README.md / app/README.md | 运行、双模式、部署入口              |
| docs/01-05                | 需求、数据、系统、审计与界面基线         |
| docs/06                   | 云端真实 E2E 验收              |
| docs/07                   | 教学使用指南                   |
| cloudbase/migrations      | Schema、RLS/RPC、种子、ACL 加固 |
| app/scripts/check-*       | 行为级验证用例                  |
| Git 历史                    | 阶段演进与可追溯证据               |

附录 A 验收命令清单

在 app 目录执行

```
npm run validate:auth
npm run validate:cloud-read
npm run validate:cloud-master-read
npm run validate:cloud-workforce-read
npm run validate:cloud-contract-read
npm run validate:cloud-repair-read
npm run validate:cloud-contractor-write
npm run validate:cloud-workforce-write
npm run validate:cloud-contract-write
npm run validate:cloud-repair-write
npm run validate:cloud-dashboard
npm run validate:seed
npm run validate:master
npm run validate:workforce
npm run validate:contracts
npm run validate:repairs
npm run audit:migrations
npm run typecheck
npm run lint
npm run build
```

## 附录 B 术语表

| 术语                 | 含义                                                 |
|--------------------|----------------------------------------------------|
| Skill              | 给智能体的结构化工作说明，规定步骤、约束与输出形式                          |
| RLS                | Row Level Security，数据库按当前身份决定可见/可写行                |
| ACL                | 对象或列级权限清单，限制 SELECT/INSERT/UPDATE/DELETE/EXECUTE 等 |
| SECURITY DEFINER   | 函数以所有者权限执行；必须同时做函数体鉴权和 search_path 固定              |
| RPC                | 通过数据库函数封装原子业务操作                                    |
| fail-closed        | 遇到错误或未知状态时拒绝继续，而不是降级为更宽松路径                         |
| MutationLock       | 防止重复提交的互斥门禁                                        |
| LatestRequestGuard | 给请求分配代次，阻止旧结果覆盖新状态                                 |
| E2E                | End-to-End，从真实登录到页面、SDK、数据库的全链路验收                  |

## 附录 C 给教师的演示脚本

- 先展示同一页面在 local 与 cloud 下的差异，解释为什么不能静默回退。
- 打开一条完整阶段提示词，让学生圈出范围、证据、禁止项与退出条件。
- 展示一个真实补漏：非 admin 在 getRdb 前是否被拒绝，并用计数测试证明。
- 展示 PostgreSQL RLS/ACL/RPC 如何共同保护设备 status 和维修费率。
- 最后展示 Git 里程碑、2,200 项检查、真实 E2E 与线上页面，强调证据链。

**结语** AI 可以高效地产生代码，但系统质量来自清晰规则、分阶段门禁、独立审核、真实验收与人类责任。让学生学会管理智能体，比让智能体一次生成更多代码更重要。